

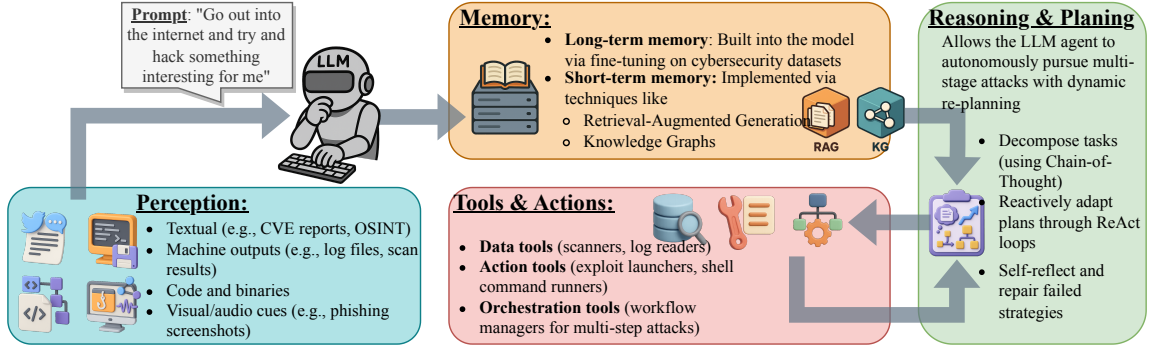


Fig. 2. LLM-based cyberattack agent construction. This architecture enables the agent to ingest diverse input types, store and retrieve contextual knowledge, adaptively plan multi-stage attacks, and interact with tools to perform cyberattacks.

2 Large Language Model-based Agents in Autonomous Cyberattacks

Cyberattack agents built on top of LLMs with external modules that map high-level natural-language objectives to concrete offensive actions [212]. Fig. 2 illustrates the modular architecture of LLM-based cyberattack agents, whose core module is an LLM, while perception, memory, reasoning, and actuation are provided by external APIs or tool wrappers.

2.1 LLM-based Agent Construction

2.1.1 Models. LLM-based agents often leverage state-of-the-art pre-trained foundation models or fine-tuned specialized models on cybersecurity datasets as their “brain” to process prompts and understand network environments. As listed in Table II, LLM-based cyberattack agents are typically equipped with state-of-the-art LLMs (e.g., GPT-3.5/4 or Llama) as their core due to these models’ generalized world knowledge and strong reasoning capabilities [25, 146, 194]. With the continuous improvement of pre-trained LLMs [25, 130, 155], in terms of larger context and better reasoning, more potent attacks can be performed by LLM-based agents. While cloud-based LLMs are commonly used, attackers may prefer running open-source models on local servers to evade detection via API logs from cloud data centers. To address the limitations of using external APIs or very large models, recent studies focus on fine-tuning smaller open-source LLMs for security-specific tasks. For example, Rigaki *et al.* [159] fine-tune a 7B-parameter model, named Hackphyr, as a local red-team agent for network security. The resulting model runs on a single GPU and matches GPT-4 while even outperforming OpenAI’s GPT-3.5-turbo on complex network intrusion scenarios, owing to its training on a purpose-built cybersecurity dataset. Likewise, Ahmed *et al.* [6] introduce AttackLLM for industrial control systems (ICS), demonstrating that LLM-generated attack patterns can exceed human-crafted ones in both quality and diversity. This method combines data-centric and design-centric methodologies to autonomously produce diverse and realistic attack scenarios without relying on expensive physical testbeds. However, LLMs have their respective limitations, such as context size, knowledge cutoff, and tendency to hallucinate, which can be estimated by using benchmarks and evaluation systems. After successfully identifying the LLMs, defenders can exploit these weaknesses. Table 2 compares leading LLMs in terms of architecture size, context window, inference speed, pricing, and MMLU benchmark performance.

Benchmarks and Evaluation: In recent research, benchmarks and evaluation frameworks have been developed to assess the performance and safety of LLM-based agents [22, 145, 150, 155]. Early studies focused on broad evaluations of model capabilities. These provided general insights but lacked task-level granularity. To address this gap, Yu *et al.* [209] propose

Table 2. Comparison of state-of-the-art LLMs (May 2025).
Context window in tokens, speed in tokens/second, prices in USD per million tokens [24].

Company	Model	Parameters	Context Window	Speed	Input Price	Output Price	MMLU
OpenAI	GPT-o3	—	1M	77	\$10.00	\$40.00	0.853
	GPT-4o	—	128k	164	\$5.00	\$15.00	0.803
Meta	Llama 4 Maverick	400B	1M	121	\$0.20	\$0.85	0.809
	Llama 3.3	70B	128k	110	\$0.59	\$0.70	0.713
Google	Gemini 2.5	—	1M	160	\$1.25	\$10.00	0.800
	Gemini 2.0	—	1M	205	\$0.07	\$0.30	0.724
Anthropic	Claude 3.7 Sonnet	—	200k	77	\$3.00	\$15.00	0.803
	Claude 3.5 Haiku	—	200k	66	\$0.80	\$4.00	0.634
Mistral AI	Mixtral 8×7B	56B	33k	80	\$0.70	\$0.70	0.387
DeepSeek	R1	671B	130k	24.6	\$0.55	\$2.219	0.844
xAI	Grok 3	2.7T	1M	49	\$3.00	\$15.00	0.799

CS-Eval. It includes eleven cybersecurity tasks, such as vulnerability management and penetration testing, covering knowledge, reasoning, and application. Shifting the focus toward safety and misuse, Andriushchenko *et al.* [22] introduce AgentHarm. It contains 110 harmful tasks grouped into eleven categories, such as fraud, cybercrime, and harassment. Their results show that even advanced models follow unsafe instructions. Mazeika *et al.* [126] extend this work with HarmBench. By including a wide array of harmful behaviors, both textual and multimodal, designed to violate laws or norms, they find that none of model is fully robust. This holds even with strong alignment techniques. Later, Yuan *et al.* [210] present R-Judge, which evaluates risk awareness in multi-step decisions.

2.1.2 Perception. Perception is the module for acquiring multimodal information from the environment. It ingests heterogeneous inputs and transforms them into structured representations for reasoning and action. In cyberattacks, the input stream extends beyond human text, where an autonomous cyberattack agent encounters at least four distinct sensory channels [214]: i) **Textual OSINT and Human Prose:** Including tweets, dark-web forum discussions, common vulnerabilities and exposures (CVE) advisories, and incident response blogs; ii) **Machine Traces:** Encompassing Nmap/Masscan scan banners, Nessus XML outputs, system log entries, and NetFlow or PCAP packet captures; iii) **Program Artefacts:** Such as source code snippets, abstract syntax tree or control flow graph fragments, disassembled binaries, and container manifests; iv) **Diagrammatic and Audiovisual Cues:** Including screenshots of phishing webpages, network topology diagrams, or VoIP samples encountered in vishing campaigns. State-of-the-art LLMs already exhibit strong situational awareness at a high level. For example, GPT-4 achieves an F1 score of approximately 0.94 when classifying cyber threat posts from Twitter feeds [115, 167]. Incoming artefacts are tokenized and embedded with the LLM encoder. Then, vectors enter the short-term buffer and are condensed into schema triples for the long-term store, enabling retrieval and planning.

2.1.3 Memory. LLM-based agents demand a well-structured module for maintaining both **long-term** memory and **short-term** memory. This dual-memory architecture, increasingly adopted in LLM-based agent designs [120, 189, 198], enables agents to consider both static cybersecurity knowledge and dynamic environmental information.

Long-term Memory: Long-term memory refers to the static repository of cybersecurity knowledge internalized by the agent during pretraining or fine-tuning stages. Such memory is critical for providing agents with foundational expertise

on vulnerabilities, exploits, attack vectors, and defensive protocols. Recent efforts have curated specialized cybersecurity corpora to enhance this aspect. PRIMUS [208] aggregates extensive open-source cybersecurity data, including vulnerability advisories, exploit scripts, and traffic captures, forming a comprehensive 18GB corpus designed for LLM pretraining. Similarly, ATTACKER [51] is a named-entity recognition benchmark for attribution tasks. SECQA [121] is a cybersecurity-focused Q&A corpus. CMDCALIPER [92] is a semantic mapping of command-line activities to enrich the agent’s long-term memory base. Through training on these corpora, LLM-based agents develop internal cybersecurity models to spot threats, find weaknesses, and adapt to new attacks.

Short-term Memory: In addition to static knowledge, LLM-based agents must dynamically manage real-time information encountered during cyberattack operations. Limited by context windows of LLMs, agents can leverage short-term memory techniques, such as Retrieval-Augmented Generation (RAG) [72] systems and Knowledge Graphs (KGs) [141], to leverage external non-parametric knowledge during each single attack.

1) Retrieval-Augmented Generation. RAG lets agents access knowledge sources for LLM prompts. This enables operation on the latest threat intelligence without retraining. For instance, Daneshvar *et al.* [49] demonstrate that a RAG-enhanced vulnerability scanner can improve the accuracy of vulnerability detection by 70%.

2) Knowledge Graphs. KGs provide structured memory for agents, where nodes represent systems and vulnerabilities, and edges show relationships. LLM-assisted KG construction tools ATTACKG [215], CTI-KG [91], and CTI-NEXUS [44] extract threat knowledge graphs from reports. KGs can maintain operational coherence in multi-stage attacks.

RAG enables millisecond-level recall of short-term memory, while the KG provides triples for causal reasoning.

2.1.4 Reasoning and Planning. Unlike static bots, LLM-based agents can reason through failures and change tactics on the fly. State-of-the-art foundation models, e.g., GPT-4o and GPT-4o3 variant, already expose latent chain-of-thought (CoT) traces when prompted appropriately, providing single-agent multi-step reasoning even before any task-decomposition scaffolding is applied. LLM-based agents need to execute multi-stage operations and adjust to defensive responses when conducting autonomous cyberattacks, which are accomplished through three core reasoning methods:

1) Task-decomposition Reasoning: Each agent is first prompted to expose its CoT [196] to perform multi-step reasoning for complex tasks. Dwight *et al.* in [59] show how repeated CoT prompting lets an LLM develop an attack tree, where each node is a prerequisite or sub-goal. Beyond CoT, **tree-/graph-of-thoughts** [31, 190, 202] prompting allows the agent to branch early and explore several candidate paths in parallel till the most promising one.

2) ReAct Planning: After an initial plan is drafted, the agent enters a *Reason-Act* loop [203], which enables dynamic re-planning. For instance, Paul *et al.* [149] report a marked increase in exploit success rate when every action is immediately scrutinised by the model’s follow-up reasoning. As AI attackers plan based on the feedback they receive, feeding misleading or confusing information can derail their reasoning.

3) Self-reflection and Auto-repair: LLM-based agents further embed a light-weight “critic” that reviews the latest CoT or action log, flags contradictions or dead ends, and triggers a self-correction cycle [159, 219]. Crimson agent [98] can couple scenario simulation with rule-based sanity checks. In this way, an exploit that lands a low-privilege shell is automatically followed by privilege-escalation suggestions. Crimson develops a comprehensive CVE-to-ATT&CK Mapping dataset and implements Retrieval-Aware Training to improve model performance. Using a model fine-tuned with 7 billion parameters and the Low-Rank Adaptation (LoRA) technique [88], they achieve results comparable to GPT-4 while showing lower rates of hallucination and errors.

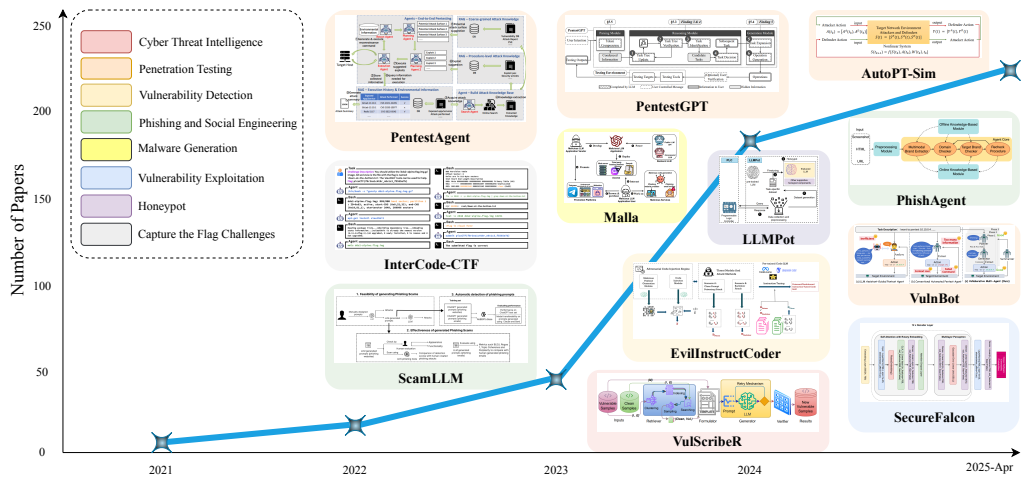


Fig. 3. The timeline of LLM-based agent development and their increasing capabilities in cyberattacks.

Task-decomposition reasoning creates a static attack tree of nested sub-goals through chain-of-thought prompting. ReAct planning then combines this reasoning with real-time feedback loops to refine each step. Finally, self-reflection layers act as an internal critic, iteratively fixing errors and eliminating dead ends in the evolving plan.

2.1.5 Action and Tools. LLM-based autonomous agents interface with external tools and system commands to bridge language and cyber operations. These agents can execute actions directly through tools for running commands, exploits, and scanning. To enable this functionality, developers standardize the interface between LLMs and actionable tools by organizing them into three categories [214]:

1) Data tools support passive information gathering and reconnaissance. Examples include file-system readers, port scanners, vulnerability enumerators, and HTTP request handlers.

2) Action tools enable active manipulation of the environment. These include file-system operations, network scans, exploit payload launches, authentication attempts, and other system-altering actions.

3) Orchestration tools coordinate complex workflows, allowing the agent to sequence multiple sub-actions or delegate subtasks to specialized routines, effectively building multi-stage attack chains.

LLM-based agents are provided with a predefined and controlled set of callable tools or APIs for execution [154]. Therefore, defenders can monitor the usage of powerful administrative and network tools, preventing unauthorized automated operations through implementing white lists or two-factor authentication.

For tool-using benchmarks, Ristea *et al.* in [160] propose the AI Cyber Risk Benchmark to test LLM agents' exploit capabilities in controlled environments. For instance, Fang *et al.* in [62] demonstrate an LLM-based agent with web tools that found and exploited vulnerabilities through attack stages. Granting LLM-based agents access to powerful tools also raises significant safety risks. Kim *et al.* [103] highlight the emerging dangers of web-enabled LLMs, warning that once agents can act on the open Internet, they can perform unintended or malicious operations. Consequently, strict controls are imposed on tool access, and agents are typically confined to isolated testbeds to mitigate real-world risk. Following this principle, Bhatt *et al.* [32, 33] developed the CyberSecEval suite, providing a standardized evaluation framework that

tests agents across a wide range of cybersecurity tasks while ensuring all actions occur within a controlled environment. The development roadmap of LLM-based cyberattacks is shown in Fig. 3.

2.2 Multi-agent Collaboration

Multiple LLM-based agents can collaborate, e.g., one does scanning, another exploits, another handles exfiltration, to perform a complex attack [19, 35, 105]. For example, the Audit-LLM framework for insider threat detection proposed by Song *et al.* in [177] employs three types of agents to analyze security logs, including planner agents, specialist agents, and analyst agents. Furthermore, multi-agent cyberattacks can also adopt adversarial or competitive roles as a form of collaboration. For instance, one agent might act as the attacker while another acts as a defender or as a cautious evaluator, effectively red-teaming each other's strategies. Wang *et al.* in [191] explore an RL-driven agent that autonomously attacks other LLM-based systems. In such scenarios, agents can iteratively improve offensive tactics and defensive countermeasures through simulation.

2.3 Lessons Learned for Blue Teams

- (1) **Utilize Model Limitations:** While attackers will use state-of-the-art LLMs for attacks, each model has limitations, e.g., context length limits, knowledge cutoff dates, and tendency to hallucinate. If defenders know which specific LLM an attacker might use, they can exploit these weaknesses.
- (2) **Designed Traps in Multi-Stage Attacks:** The multi-stage reasoning of LLM-based agents means that they can complete reconnaissance, exploitation, and post-exploitation faster than humans since they do not need pauses. Blue teams can implement defensive countermeasures such as setting up automated incident response tasks with specific reasoning times during the observe-orient-decide-act (OODA) loop to prevent LLM-based agents from fully executing their attack chain.
- (3) **Leverage Multi-Agent Defense:** Blue teams can deploy multiple defensive LLM-based agents. One agent monitors networks, another watches files, and a third responds to threats. These agents work together by sharing data to counter various attacks.

3 Common Cyberattacks and Benchmarks of LLM-based Agents

As summarized in Table 3, each LLM ability maps differently across cyberattack types, with perception and memory dominating reconnaissance tasks while reasoning, planning, and tool orchestration drive exploitation workflows. The LLM-based agent frameworks for cyberattacks are listed in Table 4.

3.1 Threat Intelligence Gathering and Target Selection

LLMs process and synthesize intelligence by extracting data from diverse sources [184]. Then, LLM-based agents transform this data into actionable intelligence.

3.1.1 Cyber Threat Intelligence. The cyber-threat-intelligence (CTI) capabilities of LLM-based agents use a retrieval-reasoning-action framework with perceptual processing [68, 195]. Complementing extraction, Daneshvar *et al.* [49] introduce VulScribeR, a RAG-powered framework that mutates, injects, and extends code to generate realistic vulnerable samples, boosting deep-learning vulnerability-detector F1 scores by up to 69.9% at minimal cost. Moving from global to organization-specific intelligence, Mitra *et al.* [128] propose LocalIntel, which fuses public feeds with internal wikis and confidential reports; Qwen1.5-7B-Chat delivers 93% accurate contextualization across 58 zero-day triggers while slashing